

Задание

Разработать универсальный двумерный контейнер `Grid<T>` для карт автономного мобильного робота

Робот работает с несколькими прямоугольными картами одинакового размера. Каждая карта хранит значения одного типа, но структура координат и базовые операции одинаковые

Размер всех карт в демонстрационном сценарии:

```
width = 8
height = 6
```

Использовать следующие карты:

Карта	Тип значения	Назначение
obstacleMap	bool	занята ли клетка препятствием
distanceMap	double	расстояние от стартовой позиции
costMap	int	стоимость прохода через клетку
probabilityMap	float	вероятность успешного проезда

Координаты клетки задаются парой x, y . Координата x находится в диапазоне от 0 до $width - 1$. Координата y находится в диапазоне от 0 до $height - 1$

Для `Grid<T>` необходимо хранить:

- ширину
- высоту
- значения клеток

Значения можно хранить в одном линейном контейнере. Отображение координат x, y в индекс линейного контейнера определить самостоятельно

`Grid<T>` должен поддерживать базовые операции для любого типа клетки:

Имя метода	Назначение
width	получить ширину карты
height	получить высоту карты
empty	проверить отсутствие клеток
contains	проверить, находится ли координата внутри карты
at	получить значение клетки по координатам
set	изменить значение клетки по координатам
fill	заполнить всю карту одним значением
print	вывести карту в поток

Для сравнимых типов клетки добавить операции:

Имя метода или функции	Назначение
countValue	посчитать количество клеток с заданным значением
findFirst	найти первую клетку с заданным значением

Для числовых типов клетки добавить операции:

Имя метода или функции	Назначение
minValue	найти минимальное значение карты
maxValue	найти максимальное значение карты
averageValue	вычислить среднее значение карты
replaceBelow	заменить значения ниже заданной границы

Операции для сравнимых и числовых типов должны быть ограничены через `concepts` и `requires`. Например, числовые операции должны быть недоступны для карты, тип клетки которой не является числовым

В программе должны быть явно созданы объекты:

```
Grid<bool>
Grid<double>
Grid<int>
Grid<float>
```

Карта препятствий должна содержать не менее 8 клеток с препятствиями. Карта стоимости должна содержать значения от 1 до 9. Карта расстояний и карта вероятностей должны содержать вещественные значения

Программа должна выполнить следующий сценарий:

- создать все четыре карты
- заполнить карты начальными значениями
- изменить несколько клеток через координаты
- вывести все карты
- посчитать количество препятствий на `obstacleMap`
- найти первую клетку с препятствием на `obstacleMap`
- вывести минимум, максимум и среднее значение для числовых карт
- заменить слишком низкие вероятности на 0.25 в `probabilityMap`
- вывести обновленную карту вероятностей

Template definition класса `Grid<T>` и его template member functions должны быть доступны в заголовочном файле

При желании можно добавить `constexpr` для небольших функций, которые проверяют размеры, координаты или вычисляют линейный индекс

Полноценный интерактивный CLI реализовывать не требуется. Достаточно подготовить в `main` сценарий с картами из задания

Требования

- Названия переменных, функций, типов и других сущностей программы должны быть осмысленными и записываться на английском языке без транслитерации
- Основной тип карты должен быть реализован как class template `Grid<T>`
- В программе должны быть созданы карты `Grid<bool>`, `Grid<double>`, `Grid<int>` и `Grid<float>`

- Template definition класса `Grid<T>` и его template member functions должны находиться в заголовочном файле
- Операции для сравнимых типов должны быть ограничены через concept или requires
- Операции для числовых типов должны быть ограничены через concept или requires
- Программа должна демонстрировать вызовы методов, доступных для любого `T`, для сравнимого `T` и для числового `T`
- Программа должна быть грамотно разбита на модули и содержать как минимум один `.hpp` файл и два `.cpp` файла. Допускается использовать больше файлов
- Для заголовочных файлов использовать `#pragma once`
- Программа должна компилироваться и выполняться без ошибок
- Студент должен понимать написанный код, уметь обосновать свое решение и отвечать на вопросы во время защиты лабораторной работы

Минимальный набор методов и функций:

- `width`
- `height`
- `empty`
- `contains`
- `at`
- `set`
- `fill`
- `print`
- `countValue`
- `findFirst`
- `minValue`
- `maxValue`
- `averageValue`
- `replaceBelow`